Test Lead

28 October 2024 08:38

Question	Response
Role as Senior Associate at BNY Mellon	**Project**: NetX 360 **Responsibilities**: Led a testing team for equity, options, complex options, and mutual fund modules. **Achievements**: Automated critical test cases (20% increase in coverage, 30% reduction in time). Implemented defect tracking (50% reduction in critical defects). Mentored junior team members and exceeded automation scorecard targets.
Strategies for High-Quality Test Coverage	- **Risk-Based Testing**: Prioritized based on criticality and impact. - **Test Data Management**: Ensured comprehensive coverage. - **CI/CD Integration**: Automated testing in the pipeline. - **API Usage**: Created test data using APIs.
Ensuring Sprint Goals & Automation Targets	- **Agile Framework**: Daily Scrums and retrospectives. - **Scrum Board**: Tracked tasks and progress. - **Automation Scorecard**: Monitored execution and failures. - **Collaboration**: Addressed blockers through teamwork or expert assistance.
Handling Difficult Test Prioritization Decisions	Scenario: During a critical phase of a NetX 360 release, a high-severity defect was identified in the complex options trading module. The defect caused incorrect pricing calculations for certain option strategies, impacting the accuracy of customer quotes and potential trades. This issue emerged just before the scheduled release, creating a significant challenge for the team. Approach: Assess Impact and Severity: We immediately assessed the severity of the defect, considering its potential impact on customer transactions and the overall system stability. Prioritize Tasks: Given the critical nature of the defect, we prioritized its resolution over other ongoing regression testing activities. We re-allocated resources to a dedicated team to focus on fixing the defect. Allocate Skilled Resources: We assigned our most experienced developers and testers to work on the defect fix. This ensured a quick and effective solution. Communicate Decisions: We communicated the situation to the project manager, development team, and other stakeholders. We provided regular updates on the progress of the defect fix and its impact on the release timeline. Monitor Progress: We closely monitored the progress of the defect fix and regression testing. We implemented a rigorous testing strategy to ensure the fix did not introduce new defects. By taking these steps, we were able to successfully address the critical defect and release the product on time, minimizing the impact on our customers. Key Learnings: Effective Communication: Clear and timely communication is crucial in managing critical issues. Prioritization: Prioritizing tasks based on impact and urgency is essential in high-pressure situations. Resource Allocation: Allocating skilled resources to critical tasks can significantly accelerate resolution. Rigorous Testing: Thorough testing is essential to ensure the quality and reliability of the product.
Deal with a conflict within your team. How did you resolve the situation and maintain a positive working environment	During a particularly busy sprint on the NetX 360 project, one team member expressed concern about their workload, feeling they were consistently taking on more tasks than others. This led to some tension within the team and impacted overall morale. To address this issue, I initiated a team meeting to openly discuss the workload distribution. I encouraged each team member to share their perspective and any challenges they were facing. After listening to everyone's concerns, I analyzed the team's workload and identified areas where tasks could be redistributed. I implemented a more transparent task allocation process, ensuring that tasks were assigned based on individual strengths, workload, and skillset. I also encouraged team members to communicate openly and proactively if they felt overwhelmed. By addressing the issue directly and taking steps to improve workload distribution, I was able to maintain a positive and productive team environment." Key points highlighted:

	Active listening: Demonstrates your ability to understand team members' concerns. Transparent communication: Shows your commitment to open and honest communication. Fair workload distribution: Highlights your ability to ensure equitable task allocation. Empathy: Emphasizes your understanding of team members' feelings and needs.
How do you manage flaky tests in an automation framework to ensure reliable execution, especially in CI/CD pipelines	When tests are flaky in CI/CD pipelines, despite running smoothly in local environments, I first investigate root causes through enhanced logging and screenshots to capture runtime details. Often, flakiness in headless mode arises from resolution or element load issues, which I address by explicitly setting the browser resolution and using adaptive waits. I also consider retry mechanisms, adding waits and test isolation strategies to further stabilize execution
Can you describe a time when you had to adapt to a significant change in project requirements or timelines? How did you handle the situation, and what steps did you take to ensure the success of the project	During a recent sprint on NetX 360, we encountered a last-minute change request regarding complex options trading functionality. The change involved a new order type for butterfly spreads, which significantly impacted our existing test suite for complex options. Initially, the change seemed daunting as it required revising test cases for butterfly spreads across various underlying assets and strike prices. To ensure project success, I followed a structured approach: 1. Assess the Impact: I collaborated with the development team to understand the technical details of the new order type. We then analyzed the impact on existing test cases, prioritizing those critical for butterfly spreads and identifying areas that needed modification. 2. Reprioritize Tasks: While maintaining regression coverage for other functionalities, we prioritized the development and execution of new test cases for butterfly spreads. This involved working with the team to adjust the sprint backlog and allocate resources effectively. 3. Leverage Automation: To expedite testing and minimize delays, we leveraged our existing automation framework. We created reusable test components for common functionalities within complex options trading, allowing us to quickly adapt our automation scripts for butterfly spreads. 4. Communication and Collaboration: I kept all stakeholders informed of the change and its impact on the sprint schedule. We held daily stand-up meetings to discuss progress and address any roadblocks collaboratively. 5. Continuous Monitoring: Throughout the sprint, I closely monitored the execution of both new and existing test cases. This allowed us to identify and address any issues promptly, ensuring the quality and functionality of the new order type while adhering to the revised timeline
Can you describe a time when you had to make a difficult decision that had a significant impact on the project? What were the factors you considered, and how did you communicate your decision to the team	During a critical phase of the NetX 360 project, we faced a significant challenge: a critical defect was discovered in the core trading engine, impacting multiple functionalities. This defect had the potential to delay the release and cause significant financial losses. Given the severity of the issue and the tight deadlines, I had to make a difficult decision: whether to prioritize the defect fix and risk delaying the release, or to proceed with the release and address the defect in a subsequent hotfix. I considered the following factors: Severity of the defect: The potential impact of the defect on the end-users and the overall system stability. Risk assessment: The risks associated with both options, including potential financial losses and reputational damage. Timeline constraints: The time required to fix the defect and retest the affected functionalities. Client expectations: The client's expectations for the release timeline and the impact of a delayed release. After careful consideration, I decided to prioritize the defect fix and delay the release. I communicated this decision to the team, explaining the rationale and the potential consequences.
How do you handle dynamic or frequently changing elements in Selenium to ensure your tests remain stable?	To handle dynamic or frequently changing elements in Selenium, here's a comprehensive answer: --- **Answer:** "When dealing with dynamic or frequently changing elements in Selenium, I focus on strategies that improve element identification and test stability: 1. **Use of Robust Locators:**

	I avoid brittle locators like absolute XPaths and prefer:
	- **Relative XPaths:** Use stable attributes such as `id`, `name`, or `data-*` attributes.
	``java
	WebElement element = driver.findElement(By.xpath("//button[contains(@class,'submit-btn')]"));
	``
	- **CSS Selectors:** When class names or custom attributes are reliable.
	``java
	WebElement element = driver.findElement(By.cssSelector("button[data-action='submit']"));
	``
	
	2. **Dynamic Waits:**
	Implementing waits ensures the script adapts to changes in load times:
	- **Explicit Waits:** Wait for an element's condition (e.g., visibility or clickability) to be met.
	``java
	WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
	WebElement element = wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("dynamicElement")));
	``
	
	3. **Index-Free Locator Strategies:**
	For lists or grids, instead of relying on indices, I use dynamic attributes to locate specific rows or cells:
	``java
	WebElement row = driver.findElement(By.xpath("//table//tr[td[text()='Specific Value']]"));
	``
	
	4. **Handling Randomized IDs:**
	When IDs or classes change dynamically, I identify consistent patterns (e.g., prefixes or suffixes) and use regular expressions in XPath:
	``java
	WebElement element = driver.findElement(By.xpath("//*[starts-with(@id,'dynamicPrefix')]"));
	``
	
	5. **Reusable Utility Methods:**
	I create utility methods to encapsulate dynamic handling logic, improving maintainability:
	``java
	public WebElement getDynamicElement(String attribute, String value) {
	return driver.findElement(By.xpath("//*[@" + attribute + "='" + value + "']"));
	}
	``
	
	• Environment Parity
	• Configuration Management: Environment Variables & Secret Management
	• Containerization
	• Parallel Execution
	• Monitoring and Reporting

How do you ensure that your Selenium-Cucumber framework is scalable and maintainable as the number of tests grows and the application evolves over time?	Modular Framework Design:Feature Files,Step Definitions,Reusable Utilities Page Object Model (POM) Implementation Centralized Test Data Management Environment-specific properties (e.g., URLs, credentials) are configured using a properties or configuration file. Regular Maintenance and Refactoring Confluence for common methods
How do you approach test case design for a complex application like NetX360, especially for scenarios involving multiple interactions and dependencies?	Requirement Analysis: Identify key user scenarios and use cases and breakdown complex scenario into separate cases Test Data Preparation Test Case Review Test Case Prioritization
As a QA Lead, how would you plan to introduce automated tests for future sprints of a food ordering web application to improve the speed of feedback cycles?	1. Define an Automation Strategy: - Choose an approach based on the application and sprint goals (e.g., Data-Driven, Modular, or Hybrid). - Focus on user stories with significant data dependencies or specific modules. 2. Implement Shift-Left Testing: - Collaborate early with developers to identify changes (e.g., validation or success messages). - Update assertions dynamically using predefined validation and success messages. 3. Phase-Wise Automation:

<table>
<tr><td></td><td>- Categorize tests into Smoke, Sanity, and Regression to prioritize based on business impact.
- Include critical business functionalities in the smoke suite to detect blocker issues early.

4. Business Context Awareness:
- Understand the application's domain (e.g., seasonal discounts during festive periods).
- Automate scenarios related to pricing, discounts, and user flows to ensure accuracy.

5. Increase Test Coverage:
- Ensure automation adds depth and breadth to the testing scope.
- Avoid automating for the sake of it; focus on business-critical and high-risk areas.

6. Resource Planning:
- Balance the team between manual and automation testers to meet sprint goals efficiently.

Outcome: Early detection of defects, quicker feedback, and improved sprint efficiency.</td></tr>
<tr><td>Test Plan</td><td>Detailed document that outlines the scope, approach, resources, and schedule of intended testing activities. It provides a roadmap for how testing will be conducted, detailing the specific tasks, who will perform them, when they will be done, and what resources are required. The test plan is tactical, focusing on the execution of the strategy by describing the "how" of testing.

Components:
<table>
<tr><td>Test Objectives</td><td>The specific goals of testing.</td></tr>
<tr><td>Test Scope</td><td>Detailed description of what features or functionalities will be tested.</td></tr>
<tr><td>Test Approach</td><td>Detailed approach to testing based on the test strategy.</td></tr>
<tr><td>Test Environment</td><td>Specific configurations and settings for the test environment.</td></tr>
<tr><td>Test Schedule</td><td>Timeline for the testing activities.</td></tr>
<tr><td>Resources</td><td>Required tools, people, and environments for testing.</td></tr>
<tr><td>Risk Management</td><td>Identification of potential risks and strategies for mitigation.</td></tr>
</table></td></tr>
<tr><td>Test Strategy</td><td>High-level document that outlines the general approach and objectives for testing. It is the "big picture" view, setting the direction and purpose for testing activities. This document typically covers the overarching goals, methodologies, tools, environments, and other factors influencing how testing will be performed.

Provides the "why" and "what" of testing. It is more static and rarely changes throughout the project unless there is a significant change in project goals or scope.

Components
<table>
<tr><td>Major Test Objectives</td><td>What are the primary goals of testing?</td></tr>
<tr><td>Scope of Testing</td><td>What is included and excluded in testing?</td></tr>
<tr><td>Types of Testing</td><td>What kinds of testing (functional, performance, security, etc.) will be conducted?</td></tr>
<tr><td>Test Environment</td><td>What environments will be used for testing?</td></tr>
<tr><td>Risks and Assumptions</td><td>What risks are associated with the testing process?</td></tr>
<tr><td>Critical Success Factors (CSFs)</td><td>Key attributes that the product must achieve to be considered successful (e.g., performance, security, reliability).</td></tr>
</table></td></tr>
<tr><td>Pairwise Testing</td><td>It is used to reduce the number of test cases while maintaining effective test coverage. It works by focusing on the interactions between different input parameters in pairs, rather than testing every possible combination of inputs (which can be impractical for systems with many parameters)

Suppose you're testing a system with 3 parameters:

Browser: Chrome, Firefox, Edge
OS: Windows, Mac, Linux
Connection: Wifi, Ethernet
If you wanted to test every possible combination, you'd need 3 × 3 × 2 = 18 test cases. With pairwise testing, you'd select test cases that cover all pairs of inputs, reducing the total number of cases while still providing good coverage. Instead of 18, you might only need something like 6-9 test cases, depending on the relationships between inputs.</td></tr>
<tr><td>ThreadLocal Concept</td><td>ThreadLocal ensures each thread gets its own WebDriver instance, avoiding conflicts in multi-threaded environments. Implement it using ThreadLocal<WebDriver> in the TestBase class and provide a getter method to access the driver.</td></tr>
</table>

Handling Async JS	Selenium uses executeAsyncScript() to run JavaScript asynchronously. For handling waits, WebDriverWait combined with ExpectedConditions helps synchronize elements.
Handling Dynamic Elements	Use dynamic locators (e.g., XPath with contains), explicit waits, and retry mechanisms to handle dynamic web elements.
Scalable Grid Infrastructure	Use Selenium Grid or Docker containers with cloud providers. Leverage CI tools like Jenkins for test parallelization across browsers.
Overcoming Selenium Limitations	Handled file uploads via Robot class and download management through browser preferences. Managed browser compatibility with remote browsers.
Page Object Model (POM)	Ensured scalability by creating modular Page Objects, grouping reusable functions, and integrating them with data-driven frameworks.
File Uploads & Downloads	Used sendKeys for uploads and customized browser preferences for handling downloads. A challenge was verifying large file downloads, resolved via checksum validation.
Debugging CI/CD Issues	Identified environment-specific issues by adding logging, taking screenshots, and adjusting wait times to align with pipeline performance.
Handling Authentication	Used OAuth 2.0 by obtaining tokens via REST requests; configured API keys in headers dynamically during tests.
Automated API Suite	Created modular API tests with Rest Assured/Postman. Integrated with Jenkins using Newman or Maven plugins for automation in CI/CD.
Schema Validation	Used JSON schema validation libraries. Ensured backward compatibility through versioning and automated schema checks.
Testing Rate Limits	Simulated concurrent requests using parallel execution and verified response codes (429).
Handling Third-Party Services	Used mock servers and stubs to decouple dependencies. Validated responses and ensured fail-safes for service outages.
Data-Driven API Testing	Leveraged parameterization with CSV/Excel files and reusable test data configurations to validate various endpoints.
Singleton & Factory Patterns	Singleton ensures one instance per class, ideal for DB connections. Factory pattern simplifies WebDriver instantiation across browsers.
Ensuring Thread Safety	Used ThreadLocal and synchronized blocks where required to maintain state across parallel tests.
Design Principles	Followed SOLID principles, ensured modularity, and created reusable components to manage complexity.
Optimizing Execution Time	Used parallel execution, reduced browser wait times, and implemented selective test runs based on tags.
Ensuring Test Reliability	Managed flakiness with retry mechanisms, wait strategies, and stabilizing the CI environment.
Using Selenium for Performance	Selenium can validate page load times but is limited for in-depth performance testing. Tools like JMeter or Gatling are preferred.
Handling Bottlenecks	Identified bottlenecks using profiling tools and optimized scripts by refactoring long wait times.
Framework Integration with CI/CD	Integrated with Jenkins/GitHub Actions, triggering tests on code commits and managing reports via Allure.
UI vs API vs Unit Testing	Balanced by prioritizing API and unit tests for faster feedback; kept critical UI tests minimal but thorough.
Managing Version Control	Used Git branching strategies to manage automation scripts and ensured smooth merges through code reviews.
Logging & Reporting	Implemented detailed logging and used reporting tools like Allure or ExtentReports for better insights.
Test Coverage & Prioritization	Prioritized high-risk features and core functionalities, using risk-based testing and test impact analysis for release cycles.
Complex Problem Resolution	Resolved multi-browser compatibility issues by leveraging capabilities management and browser drivers effectively.
Debugging Flaky Failures	Introduced stability checks, extended logging, and adjusted environment parameters to isolate flaky test behavior.
Legacy System Automation	Managed legacy systems by reverse engineering APIs, using exploratory testing, and gradually building automation.
Backend Automation: How do you test a new feature in backend automation?	Start by understanding the feature requirements, then create test cases to cover all scenarios. Use tools like REST Assured to write automated tests, execute them, and validate the outputs against expected results. Review logs and monitor the system to ensure there are no unexpected behaviors.
REST Assured Basics: Explain client-server architecture in REST Assured and the concept of method chaining.	In REST Assured, the client-server architecture involves the client sending requests to the server and receiving responses. Method chaining allows multiple methods to be called in a single statement, making the code more readable and efficient.
Comparison with Postman: Why use REST Assured for backend automation instead of Postman?	REST Assured is a Java library that integrates well with Java-based automation frameworks, offers powerful features for validating responses, and supports continuous

	integration. Postman is more suited for manual testing and initial API explorations.
Data Structures: Differences between HashMap and HashTable.	HashMap is non-synchronized and allows one null key and multiple null values. HashTable is synchronized and doesn't allow any null key or value. HashMap is generally faster than HashTable.
Serialization: What are serialization and deserialization?	Serialization is the process of converting an object into a byte stream for storage or transmission. Deserialization is the reverse process of converting the byte stream back into a copy of the object.
Memory Management in DSA: How are variables stored in memory?	Variables are stored in memory based on their type. Primitive types are stored directly on the stack, whereas objects are stored in the heap, with the stack holding references to the heap memory.
DSA Types: What are the primary data structures and how does a stack work?	Primary data structures include arrays, linked lists, stacks, queues, trees, and graphs. A stack works on LIFO (Last In, First Out) principle.
Automation Challenges: What challenges do you face during test automation?	Common challenges include handling dynamic content, managing test data, dealing with flaky tests, integrating with CI/CD pipelines, and ensuring cross-browser compatibility.
Design Patterns: What is the singleton pattern and how is it used in test automation?	The singleton pattern ensures a class has only one instance and provides a global point of access to it. In test automation, it's used to manage configuration settings, driver instances, and other shared resources.
Selenium Details: Explain iframes and the use of robot classes.	iFrames are HTML documents embedded within another HTML document. Use `driver.switchTo().frame()` to interact with them. Robot classes are used for simulating keyboard and mouse actions, useful for handling pop-ups and other non-HTML elements.
Dynamic Locators: Discuss dynamic locator strategies and axes XPath syntax.	Dynamic locators use attributes that change based on the context. Axes in XPath allow navigation through the XML tree relative to the current node, e.g., `following-sibling::`, `preceding-sibling::`.
CI/CD Experience: How have you effectively used CI/CD pipelines	Integrated automated tests into CI/CD pipelines to ensure immediate feedback on code changes. This reduced the time for regression testing and improved the overall quality of the software.
Test Strategy & Estimation: How do you prepare a test strategy and estimate for new product features?	Analyze requirements, identify test scenarios, prioritize test cases, and determine resources needed. Estimate time based on complexity and historical data. Document the strategy and review it with stakeholders.
Microservices Architecture: Explain microservice architecture, payloads, and their traces.	Microservices architecture breaks down an application into smaller, independent services that communicate over HTTP. Payloads are data sent between services, typically in JSON or XML format. Traces help in tracking requests across different services for debugging and performance monitoring.
How do you handle a stale element in Selenium?	Use a try-catch block to catch `StaleElementReferenceException` and re-locate the element. You can also implement a wait strategy to ensure the element is fully loaded before interacting with it.
Why might Selenium's hover or click not work on an element	Possible reasons: dynamic content or overlays. Solutions: use JavaScriptExecutor to perform the action or scroll the element into view using JavaScript or Selenium's Actions class.
What are essential credit card test cases for validations?	Check number length, format, and type-specific rules such as Luhn algorithm for validity, card expiration date, CVV, and name format.
What challenges might a team face when transitioning from a traditional release model to a Continuous Deployment pipeline	Challenges include managing release cycles, ensuring code quality, handling infrastructure changes, and training the team on new practices. Address these by implementing incremental changes, continuous integration, automated testing, and proper documentation.
How do deployment pipelines integrate security practices (DevSecOps), and what measures would you implement to ensure that security is not compromised during Continuous Deployment?	Integrate static and dynamic security testing, use automated vulnerability scanners, enforce code reviews, and implement access controls. Regularly update dependencies and monitor security alerts.
What considerations should be taken when implementing a multi-region deployment strategy to ensure availability and minimize latency?	Consider network latency, data consistency, failover mechanisms, and compliance requirements. Use load balancers, replicate databases across regions, and implement disaster recovery plans.
Imagine a scenario where a deployment to production fails after 75% of a Rolling Deployment has been completed. What strategies would you employ to minimize service disruptions and revert to the previous stable version?	Implement canary releases and blue-green deployments. Use automated rollback mechanisms and ensure thorough testing in staging environments before production deployments.
How would you implement environment-specific configurations securely in a CD pipeline to prevent exposure of sensitive data	Use environment variables, secret management tools like HashiCorp Vault, and encrypt sensitive information. Ensure access controls are in place to limit who can view or change configuration settings.
Explain how you would use feature flags in conjunction with Continuous Deployment to manage the release of new features without impacting the entire user base.	Use feature flags to toggle features on or off without deploying new code. Roll out features gradually to a subset of users and monitor performance before a full release.
How would you design a CD pipeline for a microservices architecture to ensure that interdependencies between services are handled properly during deployment?	Use orchestration tools like Kubernetes to manage microservices. Implement service discovery and dynamic configuration management. Ensure backward compatibility and use integration tests to validate inter-service communication.
Test Design Patterns	Techniques for creating effective and maintainable test cases.

	FIRST: A mnemonic for writing good unit tests: Fast, Isolated, Repeatable, Self-validating, Timely (run frequently). Arrange-Act-Assert (AAA) Pattern: A pattern for structuring unit tests: Arrange the test scenario, Act on it, and Assert the expected outcome. Page Object Model A design pattern for organizing UI test automation code, separating test logic from page-specific elements.

How would you validate the dynamic structure of a Gmail API response where the number of unread emails varies?	When working with dynamic API responses, like those from the Gmail API where the number of unread emails can vary, it's crucial to implement robust validation strategies. Here are some key approaches to ensure the integrity of your data:
	
	1. Schema Validation:
	
	JSON Schema: Define a JSON schema that outlines the expected structure of the response, including the messages array and its individual elements.
	
	Validation Libraries: Use a library like org.everit.json.schema to validate the response against the schema.
	
	import org.everit.json.schema.Schema;
	import org.everit.json.schema.ValidationException;
	import org.json.JSONObject;
	
	// ... (code to fetch the API response as a JSON string)
	
	JSONObject responseJson = new JSONObject(responseString);
	
	// Load the JSON schema from a file or string
	Schema schema = Schema.parse(new JSONObject(schemaString));
	
	try {
	  schema.validate(responseJson);
	  System.out.println("Response is valid");
	} catch (ValidationException e) {
	  System.err.println("Validation error: " + e.getMessage());
	}
	
	2. Data Type Validation:
	
	Type Checking: Verify that each field in the response has the expected data type (e.g., string, integer, boolean).
	Range Checks: Ensure that numerical values fall within specific ranges.
	Format Validation: Validate date and time formats, email addresses, and other structured data.
	
	3. Content Validation:
	
	Required Fields: Check for the presence of mandatory fields in each message object.
	Field Values: Validate the content of fields like id, threadId, labelIds, snippet, and historyId.
	Data Consistency: Verify that related data (e.g., sender, recipient, subject) is consistent within each message.
	
	4. Response Status and Error Handling:
	
	HTTP Status Codes: Check for expected status codes (e.g., 200 for success).
	Error Messages: Parse error messages to identify specific issues and take appropriate actions.
	Retry Logic: Implement retry mechanisms for transient errors or rate limits.

Testing Rate-Limited APIs	APIs often have rate #limits to prevent abuse. Automation scripts can simulate excessive requests and validate the API's response when the limit is exceeded. Example: Sending more than 100 #requests in a minute and checking if the response returns 429 Too Many Requests.
Authentication vs **Authorization**	**Authentication**: Testing scenarios where users login via OAuth, JWT, or other mechanisms. Example: Validating token-based #authentication for different user roles using automation scripts in Selenium or Cypress **Authorization**: Verifying that only users with the correct permissions can access certain APIs. Example: #Automating role-based access tests, ensuring unauthorized users receive proper error messages.
Deployment Strategies	**Blue-Green Deployment**: Two environments (Blue and Green) are maintained. The new version is deployed to the #inactive environment, and traffic is switched only

	after successful testing. Example: Switch from Blue (live) to Green (new version) with zero downtime. - >**Canary Deployment**: Gradually roll out a new version to a subset of users while monitoring its #performance. If stable, the update is rolled out to the rest. Example: Release the update to 10% of users, monitor, and expand if no issues arise. - >**Rolling Deployment**: Incrementally replace the old #version with the new one across servers, ensuring no downtime but slower rollouts. Example: Update 20% of servers at a time, ensuring minimal impact. - >**Recreate Deployment**: The current version is fully taken down before deploying the new one. Simple but involves #downtime. Example: Take down the app, deploy the new version, and bring it back online. - >**A/B Testing**: Similar to canary, but serves different user experiences (A and B) to compare #performance and user #feedback. Example: Test new features on version B for a specific user group
How does Java's garbage collection mechanism work to manage memory efficiently?	Java's garbage collector automatically identifies and deallocates objects that are no longer in use to free up memory, ensuring optimised performance and preventing memory leaks.
Why do we use interfaces in Java	Abstraction: Provides a way to achieve abstraction. Interfaces allow us to define methods that should be implemented by classes. Multiple Inheritance: Java does not support multiple inheritance in classes, but a class can implement multiple interfaces. Loose Coupling: Interfaces help in reducing the coupling between the classes. Polymorphism: Interfaces allow us to use different implementation classes through a common reference type.
What all status codes have you encountered while testing the APIs, also explain them briefly	200 OK: The request was successful. 201 Created: The request was successful, and a resource was created. 204 No Content: The request was successful but there is no representation to return (that is, the response is empty). 400 Bad Request: The request could not be understood or was missing required parameters. 401 Unauthorized: Authentication failed or user does not have permissions for the desired action. 403 Forbidden: Authentication succeeded but authenticated user does not have access to the resource. 404 Not Found: The requested resource could not be found. 500 Internal Server Error: An error occurred on the server. 502 Bad Gateway: The server was acting as a gateway or proxy and received an invalid response from the upstream server. 503 Service Unavailable: The server is currently unavailable (overloaded or down).
What are the different ways to declare a string in Java?	1. String literal: String s = "Hello"; 2. Using new keyword: String s = new String("Hello");
How is memory allocated in String literal and New String class object?	String literals are stored in the String pool, while new keyword creates objects in the heap.
What is interface and abstract class in Java?	Interface: Defines a contract with abstract methods. Can't have concrete method implementations. Abstract class: Can have both abstract and concrete methods. Can have constructors.
Why constructor is allowed for an abstract class?	Constructors initialize the state of an object, even if it's abstract. They're used for initializing instance variables and invoking other constructors.
What is the use of the finally block?	The finally block always executes, regardless of exceptions, to release resources like file handles or database connections.
What are commonly used interfaces in Selenium?	WebDriver, WebElement, TakesScreenshot, JavascriptExecutor, Alert, WindowHandles
What is the difference between findElement and findElements?	findElement returns the first matching element, while findElements returns a list of all matching elements.
How do you handle different windows in Selenium?	Use getWindowHandles() to get all window handles. 2. Switch to a specific window using switchTo().window() with the desired handle.
Difference between 401 and 403 status codes?	401 Unauthorized: Access is denied due to invalid credentials. 403 Forbidden: Access is forbidden, even with valid credentials.
What is the return type of the getWindowHandles() method?	Set<String>
How your day to day activities look like	• Scrum Stand-ups

<table>
<tr><td></td><td>• Design test cases and Identify Test Scenarios of new Stories
• Execution of Tests: Manual+Automation scenario
• Work with Developers and BA for understanding and approvals
• Defect Management
• Guide team members</td></tr>
<tr><td>Amount gets debited but transaction failed</td><td>• Is this a daily issue?
• Does this occurs on specific accounts: Check api response,DB value, might be ui issue
• Does this occurs on specific time:What time, any updates happening during that time</td></tr>
<tr><td>Incorrect Interest Calculation for savings account</td><td>• Check in QA env to replicate the issue
• Calculation might fail in leap year
• If only fails for customer: Check cache, browser version and app version
• Revisit the requirement and understand have we missed something</td></tr>
<tr><td>Test Plan for e-commerce website</td><td>Purpose of test Plan: Cover functional, security, performance and usability
Define the Test Approach, scope of testing, data security, acceptable performance
Scope: Modules to be tested like search,filter etc
Approach:CBT, CDT, performance usability
Env Set up
Tools to be used
Test artifacts to be provided at the end
Entry and exit criteria
Schedule
Roles and responsibilities
Risks</td></tr>
<tr><td>How much time to test search functionality of ecommerce site</td><td>• Test Case design
• Test data
• Share TC and TS
• Execution
• Search product with all types of filter available like red shirt, medium shirt etc</td></tr>
<tr><td>How to estimate a story</td><td>• Complecity, are test cases available?
• Is functionality stable
• Are page classes available
• No of test cases to be automated</td></tr>
<tr><td>Re run failed tests</td><td>In cucumber.prop as plugin
cucumber.plugin=rerun:target/rerun.text
In original test runner add this plugin
Plugin={"rerun:target/rerun.text"}

Create a new test runner where features="target/failedrerun.txt"</td></tr>
<tr><td>Difference between oauth 1 and 2</td><td><table>
<tr><th>Aspect</th><th>OAuth 1.0</th><th>OAuth 2.0</th></tr>
<tr><td>Specification</td><td>Complex, older specification.</td><td>Simplified, modern specification.</td></tr>
<tr><td>Security Model</td><td>Relies on cryptographic signing (HMAC-SHA1 or RSA-SHA1).</td><td>Uses bearer tokens with optional encryption and transport-layer security.</td></tr>
<tr><td>Ease of Use</td><td>Difficult to implement due to signature requirements.</td><td>Easier to implement with no need for cryptographic signing.</td></tr>
<tr><td>Token Type</td><td>Access tokens require cryptographic validation.</td><td>Access tokens are bearer tokens (anyone with the token has access).</td></tr>
<tr><td>Authorization Grant Types</td><td>Limited to a single flow.</td><td>Supports multiple grant types (e.g., Authorization Code, Implicit, etc.).</td></tr>
<tr><td>Access Token Exchange</td><td>Tokens exchanged via signed HTTP requests.</td><td>Tokens exchanged over HTTPS.</td></tr>
<tr><td>Client Confidentiality</td><td>Supports client applications (confidential clients).</td><td>Supports both confidential and public clients.</td></tr>
<tr><td>Backward Compatibility</td><td>OAuth 2.0 is not backward-compatible with OAuth 1.0.</td><td>Independent, not backward-compatible with OAuth 1.0.</td></tr>
<tr><td>Use Cases</td><td>Primarily used in scenarios requiring high-security communication.</td><td>Broadly adopted for web, mobile, and APIs.</td></tr>
<tr><td>Session Management</td><td>No standardized session management.</td><td>Offers token expiration and refresh tokens for session management.</td></tr>
</table></td></tr>
<tr><td>Test case design in API verses UI</td><td>API execution is faster
UI has High maintenance and failure rate
UI only for E2E</td></tr>
<tr><td>How to validate 3rd party payment gateway</td><td>Mock response to test +ve and -ve cases</td></tr>
<tr><td>Contribution to automation framework</td><td>Handeled dynamic elements
Added 2 person to approve MR</td></tr>
<tr><td>Interface usage in framework</td><td>Weblement, js executor, takes screenshot</td></tr>
</table>

Usage of abstraction	Page classes and step def, in step def we only use functions and hide unnecesary details. Utility Classes
Explain Automation Framework	• Language-Java • Build-Maven • Automation Tool: Selenium, Cucumber, rest assured • Explain Pom.xml • Structure: Feature, step def, page objects, runner, utils, listners, test base, prop file, test data, reports, gitlab.yml
How to perform RCA	Env issue, app issue, service dependency
Execute 1000+ cases in 2 days	In automation run in parallel Manual: Get help from dev or other team's tester and prioritize lower test cases to other team member If they are junior
Difference between mvn clean,mvn verify ,mvn test	mvn clean: Deletes the target directory to remove previous build artifacts. mvn verify: Validates the project, runs integration tests, and checks project quality criteria. mvn test: Runs the unit tests for the project.
How to Design Test Cases for an Online Payment Gateway	Whenever asked to design test cases you can consider the below approach to cover all aspects, start from UI Tess -> functional and negative tests -> boundary cases -> security tests -> performance validation: 1. UI & Usability Tests: ☛ Test mobile responsiveness and verify layout across various devices. ☛ Ensure clarity of instructions, ease of navigation, and visibility of error/success messages. ☛ Test accessibility features, making sure the process is smooth for all users. 2. Functional Tests: ☛ Validate payment workflows for different payment methods (credit card, debit card, UPI, PayPal, etc.). ☛ Ensure successful payments are reflected accurately in the user's transaction history. ☛ Confirm that each step—like entering card details, OTP, and confirmation—functions smoothly. 3.Negative Test Cases: ☛Test invalid inputs like expired cards, incorrect CVVs, and incorrect billing addresses. ☛ Check the system's behavior with insufficient funds and blocked cards. Validate error messages and see how the system handles canceled payments and re-tries. 4. Boundary Cases: ☛ Test payments at the minimum and maximum transaction amounts allowed by the platform. ☛ Check how the system responds to network timeouts or session timeouts during payment. ☛ Verify handling of simultaneous transactions from the same account. 5. Security Tests: ☛ Ensure secure transfer protocols (e.g., HTTPS) and validate encryption of sensitive information. ☛ Test against vulnerabilities like SQL injection, XSS, and ensure no sensitive data (like card numbers) is stored insecurely. ☛ Validate user session timeouts and check logout mechanisms for security. 6. Performance & Load Tests: ☛ Simulate high traffic to assess if the gateway can handle peak loads without slowing down. ☛ Measure transaction speed to ensure there's no noticeable delay in the user experience. ☛ Check response times during multi-currency conversions if applicable.
Prioritizing test cases for automation	High-priority functionalities, repetitive scenarios, smoke tests.
Smoke vs Sanity	Smoke Testing verifies the critical functionalities of the system whereas Sanity Testing verifies the new functionality like bug fixes. Smoke testing is a subset of acceptance testing whereas Sanity testing is a subset of Regression Testing. Smoke->Sanity->Regression
How many test cases can you automate in a day	• Complexity of test cases: Simple cases (verifying messages, login) take less time to automate than complex ones (interacting with multiple systems). • Familiarity: New tools, applications, or frameworks slow down automation. • Existing framework: A well-developed framework with utilities and reporting saves time. • Data preparation and environment stability: Unstable environments or lack of test data

	require extra effort. • Experience level: Beginners automate fewer cases than experienced testers. • Tooling and integration: Using multiple tools and CI/CD pipelines adds complexity. • Manual vs. full-time automation: If you also do manual testing, you'll automate fewer cases. number of test cases I can automate in a day depends on various factors for example complexity of the test cases robustness of the existing framework stability of the test environment the Level of details in the test script for example if I have simple scenarios or simple test cases I can automate around 5 to 10 test cases in a day but for complex test cases or complex scenario this might reduce to one to two test cases per day but our goal is always maintain a balance between speed with high quality scripts and maintainable scripts
An online shop sells pet food and accessories. Customers can choose goods and conclude their orders on its website. The order is processed according to these rules: • A customer enters his or her order and clicks *SUBMIT*. Depending on its total value, the order is set to either of the following states: ◦ **APPROVED**, if the value is **less than or equal** to $100. ◦ **PENDING**, if the value is greater than $100 and the shop's personnel have to verify the validity of the online payment. • For **PENDING** orders, shop personnel can click one of the following: ◦ *APPROVE* if the payment is confirmed, thereby setting the order to the state of **APPROVED**. ◦ *CANCEL* if the payment is invalid, thereby setting the order to the state of **CANCELED**. • For orders in an **APPROVED** state, the shop personnel choose the *SHIP* action after the goods are actually sent, thereby setting the order to a state of **SHIPPED**. Prepare the optimal (effective and efficient) set of test cases, each containing either a complete or partial state lifecycle, required to test this system.	1. Order Value ≤ $100: Test Case 1: Valid Order Customer enters an order with a total value of $99. Order is automatically set to APPROVED state. Shop personnel ships the order, setting it to SHIPPED state. 2. Order Value > $100: Test Case 2: Valid Order with Approved Payment Customer enters an order with a total value of $101. Order is set to PENDING state. Shop personnel verifies the payment and approves the order, setting it to APPROVED state. Shop personnel ships the order, setting it to SHIPPED state. Test Case 3: Invalid Order with Declined Payment Customer enters an order with a total value of $150. Order is set to PENDING state. Shop personnel verifies the payment and cancels the order, setting it to CANCELED state. 3. Edge Cases: Test Case 4: Order with Zero Value Customer enters an order with a total value of $0. Order is automatically set to APPROVED state. Shop personnel ships the order, setting it to SHIPPED state. Test Case 5: Order with Maximum Value Customer enters an order with a total value of $9999. Order is set to PENDING state. Shop personnel verifies the payment and approves the order, setting it to APPROVED state. Shop personnel ships the order, setting it to SHIPPED state. Test Case 6: Order with Invalid Character Input Customer enters an order with invalid characters in the total value field (e.g., "abc"). The system should display an error message or prevent the order from being submitted. 4. Additional Considerations: Security Testing: Verify that the system is secure against unauthorized access and data breaches. Test for vulnerabilities like SQL injection and cross-site scripting (XSS). Performance Testing: Test the system's performance under heavy load to ensure it can handle a large number of orders. Measure response times and identify bottlenecks. Usability Testing: Test the user interface to ensure it is easy to use and understand. Verify that error messages are clear and helpful. Also test payment gateway for security and functional testing, although we don't have requirements for it.
In testing a login feature, you notice that entering correct credentials works in Chrome but fails in Safari. What could be the cause, and how would you troubleshoot it?	It's possible that the discrepancy in login behavior between Chrome and Safari could be attributed to various factors, including: Browser-Specific Differences: • Cookie Handling: Safari might have stricter cookie policies or caching mechanisms that could interfere with authentication tokens or session information. • JavaScript Engine: Differences in how JavaScript is interpreted and executed by Chrome and Safari's engines could lead to variations in login form submission or

	validation. • Browser Extensions: Certain extensions installed in Safari might interfere with the login process, causing unexpected behavior. Network and Security Settings: • Firewall or Proxy Settings: Network configurations in Safari might block or redirect requests to the login server, preventing successful authentication. • Certificate Issues: Safari might have stricter certificate validation rules, leading to errors when connecting to the login server. Website-Specific Issues: • Browser Compatibility: The website's code might not be fully optimized for Safari, leading to bugs or inconsistencies in login functionality. • Third-Party Library Conflicts: If the login process relies on third-party libraries, compatibility issues with Safari could arise. Troubleshooting Steps: To effectively troubleshoot this issue, consider the following steps: • Clear Browser Cache and Cookies: • Clear the cache and cookies in Safari to eliminate any stored data that might be interfering with the login process. • Check Browser Console for Errors: • Inspect the browser console in Safari for error messages or exceptions that might provide clues about the root cause. • Test in Incognito Mode: • Disable Browser Extensions: • Check Network Requests:Use the network tab in Safari's developer tools to examine the HTTP requests and responses during the login process. Look for any unusual errors or unexpected behavior. • Test on Different Devices and Networks: • Try logging in to the website on different devices or networks to isolate the issue.
You're testing a user registration form that has a date picker for selecting birth date. Users report the date sometimes shows incorrectly in the confirmation email. What could be the cause, and how would you test it?	Date Format Mismatch: The date picker might use a different date format than the one used in the confirmation email template. This could lead to incorrect parsing and display of the date. Time Zone Differences: The system might not be handling time zones correctly. If the user's time zone differs from the server's time zone, the date and time might be misinterpreted. Date Picker Bugs: The date picker component itself might have bugs or inconsistencies in how it handles date selection and formatting. Email Template Issues: The template might not be correctly rendering the date variable or might have errors in the date formatting logic. Server-Side Processing Errors: The server-side logic might be incorrectly processing or storing the date information. Testing Approach: Test with Different Date Formats: Use various date formats (e.g., DD/MM/YYYY, MM/DD/YYYY, YYYY-MM-DD) to see if the issue persists. Pay attention to how the date is displayed in the confirmation email. Test with Different Time Zones: Use a virtual machine or browser extension to simulate different time zones. Verify that the date is displayed correctly in the confirmation email. Test with Edge Cases: Use dates at the beginning and end of months and years to check for boundary conditions. Test with leap years and other special dates. Inspect Network Requests: Use browser developer tools to examine the network requests and responses. Look for any errors or inconsistencies in the data being sent to the server and received from the server. Debug Server-Side Code:

	If possible, inspect the server-side code to verify that the date is being processed and stored correctly. Check for any errors or unexpected behavior in the date handling logic. Test with Different Browsers and Devices: Test the registration form on different browsers and devices to see if the issue is browser-specific. Check Email Client Behavior: Test how the confirmation email is displayed in different email clients (e.g., Gmail, Outlook, Apple Mail). Some email clients might interpret dates differently or have specific formatting rules.
How will you test file upload functionality	

Valid File Upload	Upload a file of an allowed type (e.g., .jpg, .png, .pdf) and within the specified size limit. Verify that the file is uploaded successfully and displayed correctly.
Invalid File Type	Attempt to upload a file with a disallowed file type (e.g., .exe, .bat). Verify that an appropriate error message is displayed, preventing the upload.
File Size Limit	Upload a file that exceeds the maximum allowed size. Verify that an error message is displayed, indicating the size limit violation.
Special Characters in File Name	Upload a file with special characters in its name (e.g., file@#$%^&*().txt). Ensure that the file is uploaded correctly, and the special characters are handled appropriately.
Multiple File Upload	If the application supports multiple file uploads, try uploading multiple files simultaneously. Verify that all files are uploaded successfully and displayed correctly
Network Connectivity Issues	Simulate network interruptions or slow internet speeds. Verify that the application handles these scenarios gracefully, either by retrying the upload or displaying an appropriate error message.
Security Testing	Attempt to upload malicious files (e.g., files containing viruses or malware). Ensure that the application has robust security measures in place to prevent such uploads and protect the system.
File Download Functionality	Once files are uploaded, test the download functionality. Verify that files can be downloaded correctly and without corruption. Check if the downloaded file matches the original uploaded file
Mobile Testing	If the application supports mobile devices, test the file upload functionality on various mobile platforms (iOS, Android).
Browser and platform testing	Test on safari, linux mac

How will you test payment gateway functionality	functional Testing: Verify payments using different methods (credit card, debit card, PayPal, wallets). Ensure the payment gateway redirects to the correct page after successful/failed payments. Check Refunds,recurring payments Negative Testing: Test failed transactions (e.g., invalid card details, insufficient balance). Security Testing: Ensure sensitive data like card details are encrypted and comply with PCI DSS standards. Test OTP Performance Testing: Validate the gateway's performance during high transaction volumes, such as during sales. Cross-Browser Testing: Ensure the payment gateway works seamlessly on different browsers and devices.
Primitive Vs Non Primitive	

Feature	**Primitive Data Types**	**Non-Primitive Data Types**
Definition	Basic building blocks of data.	Created by users or provided by Java libraries.

Examples	int, char, float, boolean, byte, short, long, double	String, Array, Class, Interface, Object, List, etc.
Storage	Stores actual values directly in memory.	Stores references (addresses) to memory locations.
Size	Fixed size depending on the data type.	Size varies based on the data structure or object.
Default Values	Defaults are specific to type, e.g., 0 for numbers, false for boolean, etc.	Default is null.
Null Value	Cannot be null.	Can be assigned null.
Methods	No methods associated directly.	Methods and behaviors are defined in classes.
Inheritance	Not applicable.	Supports inheritance and polymorphism.
Memory Allocation	Allocated in the stack memory.	Allocated in the heap memory.
Performance	Faster due to direct memory operations.	Slower as they require more memory and involve references.
Mutability	Immutable (cannot be changed).	Can be mutable or immutable depending on the type.
Use Case	Used for simple values.	Used for complex structures and objects.

Test Scenarios for lift	1.Are we testing a single lift or multiple lifts? 2. What type of lift are we testing, is it a passenger lift, goods lift, service lift, express lift for VIPs, Ski lift or any other type of lift? 3. To whom does the lift cater, is it for passengers or goods? 4. Where would the lift be installed, is it going to be installed in a residential apartment or private residence? 5. How many floors will the lift intend to service? 6. What country is the lift intended to be installed in, whether the safety instructions is given in the language of the target users? 2 7. What is the capacity of the lift? 8. What are the health and safety or regulatory requirements that the lift needs to fulfill? 9. What are the weather conditions the lift has to deal with?
How would you test the appointment scheduling feature	Happy Path: Test scheduling appointments for available slots and verify confirmation notifications (email/SMS). Edge Cases: Attempt to book appointments at the same time for the same doctor to check for conflict resolution. Time Zone Testing: Verify the application handles different time zones for teleconsultations. Integration Testing: Check integration with calendar APIs (Google Calendar, Outlook) for appointment syncing. Error Handling: Verify appropriate error messages when slots are full or invalid date/time is entered.
Can we enter text without using sendKeys() ?	WrapsDriver wrappedElement = (WrapsDriver) element; JavascriptExecutor driver = (JavascriptExecutor)wrappedElement.getWrappedDriver(); driver.executeScript("arguments[0].setAttribute(arguments[1],arguments[2])", element, attributeName, value);
You are testing an e-commerce website where the price of a product fluctuates. How would you test the validation of the correct price displayed ?	Depends on the module we are validating, UI or API. Weather any changes has happened to my module or not, if yes then get test cases signed off from BA/PO. Test Cases 1. Price Fluctuations 2. Promotions 3. Currency Conversion
Non Functional Cases for Adding beneficiary	Security Test Cases: Account number masking: Verify that the account number field is masked (e.g., with asterisks or bullets) to protect sensitive information. OTP generation and validation: Enter valid data and check if an OTP is generated. Verify that the system prompts for the OTP and does not add the beneficiary without it. Enter an invalid OTP and confirm that the beneficiary is not added and an error message is displayed. Check if the OTP is not entered within the time limit, the OTP is invalidated, and the beneficiary is not added. Performance Test Cases:

	Page load time: Measure the time it takes to load each page involved in the beneficiary addition process. Concurrent access: Open the same account in two different browsers and try to add the same beneficiary simultaneously. Verify if the system handles this scenario correctly and prevents duplicate entries. Additional Considerations: Error Handling: Test how the system handles various error scenarios, such as invalid input data, network errors, or system failures. User Interface: Ensure that the user interface is clear, intuitive, and provides appropriate feedback during the beneficiary addition process. Security Best Practices: Verify that the system adheres to security best practices, such as data encryption, secure authentication, and input validation.
Misc	Cover Functional, Performance, Security, UI, Cross browser, cross device scenarios
Most Memorable Bug	Cancel Replace from holding screen was not loading data for RIA Accounts only. In SQL Date field was varchar to between was not working for inter months
Lack of Test Coverage	Identify Gaps: Determine which areas of the application are under-tested. Prioritize Test Cases: Focus on critical functionalities and high-risk areas first. Increase Automated Testing: Implement or enhance automated testing to cover more scenarios and improve consistency. Regular Reviews: Schedule regular reviews to ensure new features and bug fixes are covered by tests.
Flaky Tests	Analyze Failure Patterns: Identify and log flaky tests and analyze the patterns or common causes of failures Mock External Services: Use mocking frameworks to simulate external dependencies and reduce test flakiness. Retries and Logging: Implement retry mechanisms for known flaky tests and improve logging to diagnose failures better.
Page Object Model (POM) design pattern and its benefits	POM is a design pattern that creates an object repository for storing all web elements. It helps improve code readability, reusability, and maintainability by separating test scripts from page-specific code
Which exception is shown in selenium when there is a delay in the loading time of the elements with which we are interacting?	Without any wait: NOSUCHELEMENTEXCEPTION else TIMEOUTEXCEPTION

Suppose u have 50 apis in config out of which 48 are common to dev and test env
but 2 are diff in both. How u handle this in framework
	common.properties: Contains properties common across all environments.
	dev.properties and test.properties: Contain environment-specific overrides
	
	public class ConfigManager {
	private static Properties properties = new Properties();
	
	public static void loadConfig(String environment) throws IOException {
	// Load common properties
	try (FileInputStream commonStream = new FileInputStream("config/common.properties")) {
	properties.load(commonStream);
	}
	
	// Load environment-specific properties
	String envFilePath = "config/" + environment + ".properties";
	try (FileInputStream envStream = new FileInputStream(envFilePath)) {
	Properties envProperties = new Properties();
	envProperties.load(envStream);
	properties.putAll(envProperties); // Override common properties
	}
	}
	
	public static String getProperty(String key) {
	return properties.getProperty(key);
	}
	}

What are the important validation you will put on a API.	Important Validations for APIs When validating APIs, it's crucial to cover a wide range of aspects to ensure their reliability, security, and overall quality. Here are some key validations to consider: 1. Functional Validation: Input Validation:

	Data Type: Verify that the API accepts the correct data types for all parameters (e.g., strings, numbers, booleans). Data Format: Ensure that the data is in the expected format (e.g., date, time, currency). Data Range: Check that the values of parameters fall within acceptable ranges. Required Fields: Verify that all mandatory fields are provided. Output Validation: Data Structure: Validate that the response data structure matches the expected format (e.g., JSON, XML). Data Accuracy: Check that the values in the response are correct and consistent with the input. Status Codes: Verify that the API returns the appropriate HTTP status codes (e.g., 200 for success, 400 for bad request, 500 for server error). Error Handling: Error Messages: Ensure that the API returns meaningful error messages when invalid input is provided or unexpected errors occur. Error Codes: Verify that the API returns appropriate error codes to help identify the specific issue. 2. Performance Validation: Response Time: Measure the time it takes for the API to respond to requests under different load conditions. Throughput: Determine the number of requests the API can handle per second. Resource Utilization: Monitor CPU, memory, and network usage to identify performance bottlenecks. 3. Security Validation: Authentication and Authorization: Verify that the API properly authenticates users and authorizes access to resources. Data Encryption: Ensure that sensitive data is encrypted both in transit and at rest. Input Sanitization: Validate and sanitize user input to prevent injection attacks (e.g., SQL injection, cross-site scripting). Security Headers: Check that the API sets appropriate security headers (e.g., X-Frame-Options, Content-Security-Policy). 4. Reliability Validation: Fault Tolerance: Test the API's ability to handle failures gracefully (e.g., network outages, database errors). Availability: Ensure that the API is available when it's expected to be. Scalability: Verify that the API can handle increasing traffic load without significant performance degradation.
Explain how do you pick which test cases to automate	Choosing the Right Test Cases for Automation: A Strategic Approach When it comes to test automation, not all test cases are created equal. Careful selection is crucial to maximize the benefits and minimize the costs associated with automation. Here's a breakdown of key factors to consider when deciding which test cases to automate: 1. Frequency of Execution: High-Frequency Tests: Prioritize test cases that are executed repeatedly, such as regression tests or smoke tests. Automation can significantly reduce the time and effort required for these tests. 2. Test Stability: Stable Tests: Focus on test cases that are less likely to change frequently due to application updates or modifications. Unstable tests can lead to frequent script maintenance, increasing the overall cost of automation. 3. Business Criticality: High-Impact Tests: Concentrate on test cases that cover critical functionalities or areas with high business impact. These tests should be automated to ensure the stability and reliability of core features. 4. Test Complexity: Complex Tests: Consider automating complex test scenarios that involve multiple steps or interactions. Automation can help ensure consistent execution and reduce the risk of human error. 5. Repetitive Tasks: Manual-Prone Errors: Automate tasks that are prone to human error, such as data entry or repetitive clicks. This can improve accuracy and efficiency. 6. Return on Investment (ROI): Cost-Benefit Analysis: Evaluate the potential cost savings and time efficiency gains from automation. Prioritize test cases that offer the highest ROI.

	7. Technical Feasibility: Automation Suitability: Ensure that the test cases can be technically automated using available tools and technologies. Consider factors like the application's architecture, accessibility of elements, and the complexity of the test logic. 8. Maintainability: Long-Term Value: Choose test cases that can be easily maintained and updated over time. Avoid automating tests that are likely to become obsolete quickly.

SQL

14 December 2023 10:00

Creating a database	CREATE DATABASE mydatabase;
Using the created database	USE mydatabase;
Creating a table	CREATE TABLE employees (employee_id INT PRIMARY KEY, first_name VARCHAR(50) NOT NULL, last_name VARCHAR(50) NOT NULL, department_id INT, salary DECIMAL(10, 2) DEFAULT 0.00, CONSTRAINT fk_department FOREIGN KEY (department_id) REFERENCES departments(department_id));
Between	SELECT * FROM payment WHERE amount BETWEEN 8 AND 9; SELECT * FROM payment WHERE amount NOT BETWEEN 8 AND 9; In SQL, the BETWEEN operator's range is inclusive. That means both the start and end values of the range are included! When using between on timestamp BETWEEN '2023-01-04' AND '2023-01-05' will result in records having timestamp less than 2023-01-04 23:59:00 i.e records of 0001 HRS of 2023-01-05 will not come up. Else Use SELECT * FROM your_table WHERE timestamp_column BETWEEN '2023-01-04 00:00:00' AND '2023-01-05 23:59:59';
LIKE	The LIKE operator is used in a WHERE clause to search for a specified pattern in a column. • % - Represents zero, one, or multiple characters. • - Represents a single character. ILIKE is case insensitive SELECT * FROM customer WHERE fname LIKE '%er%' will also yield Jennifer
DISTINCT	To find unique combinations of multiple columns, such as `department` and `job_title`: SELECT DISTINCT department, job_title FROM employees; - This will return unique pairs of `department` and `job_title`.
Inserting data into the table	INSERT INTO employees (employee_id, first_name, last_name, department_id, salary) VALUES (1, 'John', 'Doe', 101, 50000.00),(2, 'Jane', 'Smith', 102, 60000.00);
Updating records	UPDATE employees SET salary = 55000.00 WHERE employee_id = 1;
Altering table structure	ALTER TABLE employees ADD COLUMN hire_date DATE;
Limiting results	SELECT employee_id, first_name, last_name FROM employees LIMIT 5;
Deleting records	DELETE FROM employees WHERE employee_id = 2;
UNION	The UNION operator in SQL is used to combine the results of two or more SELECT statements into a single result set. It removes duplicate rows by default. To include duplicates, you can use UNION ALL. The number of columns and their data types in the result set are determined by the first SELECT statement. SELECT employee_id, first_name FROM employees UNION SELECT employee_id, last_name FROM employees;
JOIN	SELECT employees.employee_id, employees.first_name, departments.department_name FROM employees JOIN departments ON employees.department_id = departments.department_id;
Find the third-highest salary	SELECT*FROM`employee` ORDERBY`salary` DESC LIMIT 1 OFFSET2;
Order by	ORDER BY in SQL sorts the resulting rows in ascending (ASC) you can substitute numbers for column names in the ORDER BY clause. The numbers correspond to the columns you specify in the SELECT clause.

Group By	When using a GROUP BY clause, any columns that are not part of an aggregate function (such as SUM, AVG, COUNT, etc.) must be included in the GROUP BY clause. This is because SQL needs to know how to group the rows when aggregating data.
	
	select customer_id,rental_id,sum(amount) from payment group by rental_id order by customer_id;//This will not work
	
	select customer_id,rental_id,sum(amount) from payment group by rental_id,customer_id order by customer_id;//this will work

	HAVING is used to filter the aggregation results. Aggregation only works in SELECT and HAVING

Case Statement	It is a conditional expression that allows you to perform different actions based on specified conditions. It's similar to the if-else construct in other programming languages.
	
	SELECT
	  id,
	  name,
	  salary,
	  CASE
	  WHEN salary >= 10000 THEN 'A'
	  WHEN salary >= 8000 AND salary < 10000 THEN 'B'
	  WHEN salary >= 6000 AND salary < 8000 THEN 'C'
	  ELSE 'D'
	  END AS salary_grade
	FROM
	  employees;

Functions	

AVG	SELECT AVG(column_name) FROM table_name
COUNT	SELECT COUNT(column_name) FROM table_name
MAX	SELECT MAX(column_name) FROM table_name
MIN	SELECT MIN(column_name) FROM table_name;
SUM	SELECT SUM(column_name) FROM table_name;
ABS	SELECT ABS(column_name) FROM table_name;
CEIL	Returns the smallest integer greater than or equal to the number SELECT CEIL(column_name) FROM table_name;
FLOOR	Returns the largest integer less than or equal to the number SELECT FLOOR(column_name) FROM table_name; Floor returns the largest integer value that is less than or equal to a given numeric expression.FLOOR(AVG(r.revenue)). Calculates the average revenue for each city and then applies the FLOOR function to get the largest integer less than or equal to that average revenue. 5.678->5
ROUND	Rounds a number to a specified number of decimal places SELECT ROUND(column_name, decimal_places) FROM table_name;
POWER	Returns the value of a number raised to the power of another number SELECT POWER(column_name, exponent) FROM table_name;
SQRT	Returns the square root of a number SELECT SQRT(column_name) FROM table_name;
CONCAT	SELECT CONCAT(string1, string2) FROM table_name;
LENGTH	SELECT LENGTH(column_name) FROM table_name;
UPPER	SELECT UPPER(column_name) FROM table_name;
REPLACE	SELECT REPLACE(column_name, 'old', 'new') FROM table_name;
EXTRACT	extracts a specific part of a date or time value in SQL -- Extracting Year: SELECT EXTRACT(YEAR FROM your_date_column) AS year FROM your_table; -- Extracting Month: SELECT EXTRACT(MONTH FROM your_date_column) AS month FROM your_table; -- Extracting Day: SELECT EXTRACT(DAY FROM your_date_column) AS day FROM your_table; -- Extracting Hour: SELECT EXTRACT(HOUR FROM your_timestamp_column) AS hour FROM your_table; -- Extracting Minute: SELECT EXTRACT(MINUTE FROM your_timestamp_column) AS minute FROM your_table; -- Extracting Second: SELECT EXTRACT(SECOND FROM your_timestamp_column) AS second FROM your_table;
TO_CHAR	The `TO_CHAR` function in SQL is used to convert a date or timestamp to a string with a specified format. This is particularly useful for displaying dates in a specific format, performing string operations, or preparing data for reports. TO_CHAR(date_value, 'format_model') SELECT TO_CHAR(SYSDATE, 'DD-MON-YYYY') AS formatted_date FROM dual; SELECT TO_CHAR(SYSDATE, 'Month DD, YYYY') AS formatted_date FROM dual; SELECT TO_CHAR(SYSDATE, 'YYYY/MM/DD HH24:MI:SS') AS formatted_date FROM dual;

As	Alias made using as cannot be used in where or having as alias are processed at the end of the query

FILTER	Used with aggregate functions like COUNT(), SUM(), AVG(), etc., to apply a condition only to the rows considered by that aggregate function.
	
	SELECT
	  COUNT(*) FILTER (WHERE device_type = 'laptop') AS laptop_views,
	  COUNT(*) FILTER (WHERE device_type IN ('tablet', 'phone')) AS mobile_views
	FROM viewership;

Netx 360 WF

26 October 2024 10:59

UI	Submit Order -> Sends data mapping to Mainframe after transforming to Mainframe's format Buy->B Sell->S Source of Input->soi
Rules Engine	Below Min amount for account/security 90 Day Trading restriction Order already exists
MDS	Quotes
Ria	Registered Investment advisor
BD	Broker Dealer
Atoms	Order Execution
M/F	Order Storage
Account Summary	Per Account All Assert Type
Order Modification	Fetch order detail from M/F based on orderdetail key
Cancel Replace	Change Qty
Correct	Update Comment, commission
BD	Extra Fields Comments, order qualifier, Commission, order date and time
Account Detail	MF
Positions balances	M/F
Entitlement	DASF

Src/main/resources	Has log4j.xml
SRC/Main/java	Has utils
SRC/Test/java	In nextgen.cucumber • Pageobjectmanager • Page classes extending basepageobject: has wait, click, send input functions,get elements after adding waits • Step Def extends abstractsteps. Abs step has start driver() which starts approp driver • Test runners extends abstracttestngcucumbertest • Bddreport listner implements itestlistner
SRC/Test/resources	• Data • Feature files • Properties • Extent.properties

Lessons Learned in Software Testing

31 August 2024 18:39

You are the headlights of the project	Testing is done to find information.
Your mission drives everything you do	Negotiating and clarifying the mission with management, and ensuring that the team's efforts are aligned with the core problems that they are responsible for
You serve many clients	PM, programmer, top management, user
You discover things that will "bug" someone whose opinion matters	Your group's mission includes (or should include) informing clients about anything that threatens the value of the product, according to your clients' definition(s) of value
Find important bugs fast	Test things that are changed before things that are the same Test core functions before contributing functions Test for high-impact problems before low-impact problems
Run with the programmers	Supporting the programmers is probably a key part of your mission. Aim for the shortest, quickest feedback loop you can
Question everything, but not necessarily out loud	Questions are fundamental to your role on the project. If you do not question, your testing will be aimless.if you ever find yourself testing and realize that you have no questions about the product, take a break
You focus on failure, so your clients can focus on success	Testers focus on failure because it improves their chances of finding it. Look for problems in the product with all your creativity and skill. By finding what's there to find in the product, you help the project team learn more about their own skills and the product's risks, and you help them make the product better, more supportable, and probably more successful in the marketplace
You will not find all the bugs	It's your job to find and report significant bugs. But you won't find all of them. To find all of them, you'd have to look everywhere there could be a bug, you'd have to look there under every different situation that could arise, and you'd need a foolproof way of recognizing every different kind of bug when it occurred
Beware of testing "completely."	Saying “it will take me five days to test that” can be interpreted to mean that you think you will have completely tested that part of the product in five calendar days. And that might be taken to mean you will find every bug in five days
You don’t assure quality by testing	Quality comes from the people who build the product
Never be the gatekeeper	when testers control the release, they also must bear the full responsibility for the quality of the product. The rest of the team will relax a little bit, or maybe a lot. If any bug sneaks by the testers and out the gate, the rest of the team can (and will) shrug and blame the testers.
Beware of the not-my-job theory of testing	Beware of the not-my-job theory of testing: Some testers feel that their mission is simply to find variances between the product and the specifications. Anything beyond that-such as usability problems, requirements problems, data quality, and supportability concerns are "not my job." We urge you to take a more expansive view. All other things being equal, your mission should be to inform the team, to the best of your ability, about any problems that could adversely impact the value of the product
Beware of becoming a process improvement group	we urge you to resist any temptation to "elevate" the test team into a process criticism society. That way lies madness
Don't expect anyone to understand testing, or what you need to do it well	It's up to you to let your clients know what you need in order to do your job effectively. An important part of the role of testing is to explain testing to our clients. Your explanations are like a flu vaccine-healthy and not that painful, but the effect wears off, so you have to do it over and over again
Testing is applied epistemology	Epistemology is a branch of philosophy that helps you test better How do you know the software is good enough? How would you know if it wasn’t good enough? How do you know you’ve tested enough?
Studying epistemology helps you test better	How to gather and assess evidence. How to make valid inferences. How to use different forms of logic. What it means to have a justified belief. Differences between formal and informal reasoning. Common fallacies in informal reasoning. Meaning and ambiguity in natural language. How to make a good decision.
Testing is grounded in cognitive psychology	cognitive psychology tells us about how we do think
Testing is in your head	The difference between excellent testing and mediocre testing is how you think: your test design choices, your ability to interpret what you observe, and your ability to tell a compelling story about it
Testing requires inference, not just comparison of output to expected results	software testing is not a simple task of comparing test cases with expected results. Instead, it requires critical thinking and creativity in test design, which involves making inferences based on available information, experience, and reasoning. Testers rarely have clear, authoritative guidelines on what should be tested or expected, so they rely on their exploratory inference skills—constantly adapting and evolving their understanding. Testing is much like scientific or mathematical reasoning, where exploration, questioning, and active problem-solving

	are key. It is not a rigid process, but one that evolves with new insights and discoveries.
Good testers think technically, creatively, critically, and practically	Thinking like a tester means recognizing that things might not be as they seem and being aware of the possibility of missing important tests due to narrow focus or tunnel vision. Effective testing involves broadening your perspective to avoid overlooking critical test areas, even when resources are limited
Black box testing is not ignorance-based testing	
A tester is more than a tourist	a tester's efforts are devoted to evaluating the product, not merely witnessing it
All tests are an attempt to answer some question	All tests are experiments performed to answer a question about the relationship between what a product is and what it should be
All testing is based on models	Effective testing depends on the quality of the models you use to design tests. By improving your skills in modeling and systems thinking, you can create more accurate and comprehensive tests, leading to better software quality